

WORKED EXAMPLE

Forward means calculate the prediction

Use the same 2-to-2-to-2-to-1 sigmoid network as Lesson16.

$$x = (1, 2), \quad t = 1, \quad y_1^{(0)} = 1, \quad y_2^{(0)} = 2$$

$$z_j^{(l)} = \sum_i w_{i,j}^{(l)} y_i^{(l-1)} + b_j^{(l)}, \quad y_j^{(l)} = \frac{1}{1 + e^{-z_j^{(l)}}}$$

The handout uses y for each neuron activation and t for the known target. z is the weighted sum before sigmoid. The bias b is the handout weight $w_{0,j}$.

Calculate both neurons of layer 1, then both neurons of layer 2, then the output. A later layer uses activations y from the previous layer, not its preactivations z .

Displayed decimals are rounded. Carry unrounded values internally, or at least 8-10 decimal places on a calculator. No parameter changes during this forward pass.

WORKED EXAMPLE

Start from the original study parameters

Weights use source index first, destination index second.

layer	w1,1	w1,2	w2,1	w2,2	b1	b2
1	0.1	-0.2	0.3	0.2	0	0.1
2	0.4	-0.3	-0.2	0.2	0.1	-0.1
3	0.3	—	-0.4	—	0.2	—

These are the original N3.4 numerical-practice values, shared with Lesson16. They illustrate the handout equations; the handout does not supply these numbers.

A dash means that destination neuron does not exist. There are 10 ordinary weights and 5 biases. All five computational neurons use sigmoid.

WORKED EXAMPLE

Layer 1, neuron 1

First multiply and add, then apply sigmoid to this neuron only.

$$z_1^{(1)} = w_{1,1}^{(1)} y_1^{(0)} + w_{2,1}^{(1)} y_2^{(0)} + b_1^{(1)}$$

$$\approx (0.1)(1.00000000) + (0.3)(2.00000000) + (0)$$

$$\approx (0.10000000) + (0.60000000) + (0) = 0.70000000$$

$$e^{-z} = e^{-(0.70000000)} \approx 0.49658530$$

$$y_1^{(1)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.49658530} = 0.66818777$$

Cache this activation for the next layer. Keep its preactivation separately; they are different quantities.

WORKED EXAMPLE

Layer 1, neuron 2

First multiply and add, then apply sigmoid to this neuron only.

$$z_2^{(1)} = w_{1,2}^{(1)} y_1^{(0)} + w_{2,2}^{(1)} y_2^{(0)} + b_2^{(1)}$$

$$\approx (-0.2)(1.00000000) + (0.2)(2.00000000) + (0.1)$$

$$\approx (-0.20000000) + (0.40000000) + (0.1) = 0.30000000$$

$$e^{-z} = e^{-(0.30000000)} \approx 0.74081822$$

$$y_2^{(1)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.74081822} = 0.57444252$$

Cache this activation for the next layer. Keep its preactivation separately; they are different quantities.

WORKED EXAMPLE

Layer 2, neuron 1

First multiply and add, then apply sigmoid to this neuron only.

$$z_1^{(2)} = w_{1,1}^{(2)} y_1^{(1)} + w_{2,1}^{(2)} y_2^{(1)} + b_1^{(2)}$$

$$\approx (0.4)(0.66818777) + (-0.2)(0.57444252) + (0.1)$$

$$\approx (0.26727511) + (-0.11488850) + (0.1) = 0.25238661$$

$$e^{-z} = e^{-(0.25238661)} \approx 0.77694431$$

$$y_1^{(2)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.77694431} = 0.56276384$$

Cache this activation for the next layer. Keep its preactivation separately; they are different quantities.

WORKED EXAMPLE

Layer 2, neuron 2

First multiply and add, then apply sigmoid to this neuron only.

$$z_2^{(2)} = w_{1,2}^{(2)} y_1^{(1)} + w_{2,2}^{(2)} y_2^{(1)} + b_2^{(2)}$$

$$\approx (-0.3)(0.66818777) + (0.2)(0.57444252) + (-0.1)$$

$$\approx (-0.20045633) + (0.11488850) + (-0.1) = -0.18556783$$

$$e^{-z} = e^{-(-0.18556783)} \approx 1.20390186$$

$$y_2^{(2)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 1.20390186} = 0.45374071$$

The preactivation is negative, so minus z is positive inside the exponential. Sigmoid still returns a positive number below 0.5.

WORKED EXAMPLE

Layer 3: the output neuron

First multiply and add, then apply sigmoid to this neuron only.

$$z_1^{(3)} = w_{1,1}^{(3)}y_1^{(2)} + w_{2,1}^{(3)}y_2^{(2)} + b_1^{(3)}$$

$$\approx (0.3)(0.56276384) + (-0.4)(0.45374071) + (0.2)$$

$$\approx (0.16882915) + (-0.18149629) + (0.2) = 0.18733287$$

$$e^{-z} = e^{-(0.18733287)} \approx 0.82916769$$

$$y_1^{(3)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.82916769} = 0.54669673$$

Cache the output probability for the loss. Keep its preactivation separately; they are different quantities.

WORKED EXAMPLE

Use the probability in binary cross-entropy

The target first enters here; it did not enter any forward neuron calculation.

$$\ell = -[t \ln y + (1 - t) \ln(1 - y)]$$

$$t = 1, \quad y = 0.5466967333$$

$$\ell = -[1 \ln(0.5466967333) + 0 \ln(1 - 0.5466967333)]$$

$$\ell = -\ln(0.5466967333) \approx 0.6038610484$$

Use natural logarithms and the probability y . Do not threshold y into class 0 or 1 before calculating cross-entropy.

This is one example loss, not an average over a batch. Changing the target would change the loss, but not this prediction at fixed input and parameters.

WORKED EXAMPLE

Keep a forward cache for backpropagation

Later derivatives reuse these activations and preactivations.

neuron	z	y = sigmoid(z)
layer1 / 1	0.7000000000	0.6681877722
layer1 / 2	0.3000000000	0.5744425168
layer2 / 1	0.2523866055	0.5627638384
layer2 / 2	-0.1855678283	0.4537407133
layer3 / 1	0.1873328662	0.5466967333

This table is a checkpoint, not a replacement for the substitutions. Lessons18-19 start from these same forward values and original weights.

Common mistakes: swapping weight indices, feeding z instead of y to the next layer, forgetting a bias, rounding too early, or changing parameters midway.

INDEPENDENT PRACTICE

Your turn: a different input and target

Reset to the same original parameter table. No training update has happened.

$$x = (-1, 1), \quad t = 0$$

Calculate all five z values and all five sigmoid activations. Then calculate the one-example binary cross-entropy.

Write each neuron as two weight-times-activation products plus its bias. Watch the negative input and the negative preactivation in hidden layer 2.

This is example B from numerical-practice N3.5. It will join example A in the later batch-update lesson.

WORKED ANSWER

Practice: layer 1, neuron 1

First multiply and add, then apply sigmoid to this neuron only.

$$z_1^{(1)} = w_{1,1}^{(1)} y_1^{(0)} + w_{2,1}^{(1)} y_2^{(0)} + b_1^{(1)}$$

$$\approx (0.1)(-1.00000000) + (0.3)(1.00000000) + (0)$$

$$\approx (-0.10000000) + (0.30000000) + (0) = 0.20000000$$

$$e^{-z} = e^{-(0.20000000)} \approx 0.81873075$$

$$y_1^{(1)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.81873075} = 0.54983400$$

Cache this activation for the next layer. Keep its preactivation separately; they are different quantities.

WORKED ANSWER

Practice: layer 1, neuron 2

First multiply and add, then apply sigmoid to this neuron only.

$$z_2^{(1)} = w_{1,2}^{(1)} y_1^{(0)} + w_{2,2}^{(1)} y_2^{(0)} + b_2^{(1)}$$

$$\approx (-0.2)(-1.00000000) + (0.2)(1.00000000) + (0.1)$$

$$\approx (0.20000000) + (0.20000000) + (0.1) = 0.50000000$$

$$e^{-z} = e^{-(0.50000000)} \approx 0.60653066$$

$$y_2^{(1)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.60653066} = 0.62245933$$

Cache this activation for the next layer. Keep its preactivation separately; they are different quantities.

WORKED ANSWER

Practice: layer 2, neuron 1

First multiply and add, then apply sigmoid to this neuron only.

$$z_1^{(2)} = w_{1,1}^{(2)} y_1^{(1)} + w_{2,1}^{(2)} y_2^{(1)} + b_1^{(2)}$$

$$\approx (0.4)(0.54983400) + (-0.2)(0.62245933) + (0.1)$$

$$\approx (0.21993360) + (-0.12449187) + (0.1) = 0.19544173$$

$$e^{-z} = e^{-(0.19544173)} \approx 0.82247127$$

$$y_1^{(2)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.82247127} = 0.54870550$$

Cache this activation for the next layer. Keep its preactivation separately; they are different quantities.

WORKED ANSWER

Practice: layer 2, neuron 2

First multiply and add, then apply sigmoid to this neuron only.

$$z_2^{(2)} = w_{1,2}^{(2)} y_1^{(1)} + w_{2,2}^{(2)} y_2^{(1)} + b_2^{(2)}$$

$$\approx (-0.3)(0.54983400) + (0.2)(0.62245933) + (-0.1)$$

$$\approx (-0.16495020) + (0.12449187) + (-0.1) = -0.14045833$$

$$e^{-z} = e^{-(-0.14045833)} \approx 1.15080113$$

$$y_2^{(2)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 1.15080113} = 0.46494303$$

The preactivation is negative, so minus z is positive inside the exponential. Sigmoid still returns a positive number below 0.5.

WORKED ANSWER

Practice: layer 3: the output neuron

First multiply and add, then apply sigmoid to this neuron only.

$$z_1^{(3)} = w_{1,1}^{(3)}y_1^{(2)} + w_{2,1}^{(3)}y_2^{(2)} + b_1^{(3)}$$

$$\approx (0.3)(0.54870550) + (-0.4)(0.46494303) + (0.2)$$

$$\approx (0.16461165) + (-0.18597721) + (0.2) = 0.17863444$$

$$e^{-z} = e^{-(0.17863444)} \approx 0.83641161$$

$$y_1^{(3)} = \frac{1}{1 + e^{-z}} \approx \frac{1}{1 + 0.83641161} = 0.54454023$$

Cache the output probability for the loss. Keep its preactivation separately; they are different quantities.

WORKED ANSWER

Practice answer: loss for target zero

Keep the probability; use the other term of binary cross-entropy.

$$y = 0.5445402309851674, \quad t = 0$$

$$\ell = -[0 \ln y + 1 \ln(1 - y)] = -\ln(1 - y)$$

$$1 - y = 1 - 0.5445402309851674 = 0.4554597690148326$$

$$\ell = -\ln(0.4554597690148326) \approx 0.7864478889$$

The model assigns more than half its probability to class 1, although the target is 0, so this is an incorrect thresholded prediction. The cross-entropy uses the probability, not just that mistake count.

Every parameter is still at its original value. Only the input, target and resulting cached quantities changed.